

ADDEVENTLISTENER

¿Qué es addEventListener?

En JavaScript, addEventListener es un método que te permite adjuntar un evento a un elemento del DOM (Modelo de Objeto del Documento). En términos más sencillos, te permite "escuchar" cuando ocurre algo en un elemento de tu página web (como un clic en un botón, un cambio en un campo de texto, el movimiento del mouse, etc.) y luego ejecutar una función específica en respuesta a ese evento.

¿Para qué sirve?

Su principal utilidad es hacer que las páginas web sean **interactivas**. Sin addEventListener, tu página sería estática, es decir, solo mostraría información sin responder a las acciones del usuario.

Con addEventListener, puedes:

- **Manejar clics de usuario:** Por ejemplo, cuando un usuario hace clic en un botón, puedes mostrar un mensaje, enviar un formulario, o cambiar el contenido de la página.
- **Reaccionar a la entrada de texto:** Puedes validar un formulario en tiempo real, mostrar sugerencias de autocompletado mientras el usuario escribe, o contar los caracteres de un área de texto.
- **Detectar movimientos del mouse:** Puedes crear efectos visuales, mostrar información contextual cuando el mouse pasa sobre un elemento, o arrastrar y soltar objetos.
- **Responder a eventos del teclado:** Puedes crear atajos de teclado, o mover un personaje en un juego.
- **Controlar la carga de la página:** Puedes ejecutar código cuando la página ha terminado de cargarse por completo.

Sus utilidades (en detalle):

addEventListener es la forma moderna y recomendada de manejar eventos en JavaScript por varias razones:

1. **Permite adjuntar múltiples funciones al mismo evento:** A diferencia de las antiguas propiedades de evento (como onclick = funcion1), con addEventListener puedes adjuntar varias funciones que se ejecutarán cuando ocurra el mismo evento. Por ejemplo, puedes tener una función que valide un formulario y otra que envíe un evento a un sistema de análisis, ambas activadas por el mismo clic del botón.

```
const boton = document.getElementById('miBoton');
```

```
// Primera función que se ejecuta al hacer clic
```

```
boton.addEventListener('click', function() {  
    console.log('Se hizo clic en el botón!');  
});
```

```
// Segunda función que también se ejecuta al hacer clic
```

```
boton.addEventListener('click', function() {  
    alert('¡Gracias por hacer clic!');  
});
```

Facilita la eliminación de oyentes de eventos: Puedes usar el método removeEventListener para eliminar una función específica que se adjuntó previamente. Esto es útil para optimizar el rendimiento y evitar fugas de memoria, especialmente en aplicaciones de una sola página (SPA).

```
const boton = document.getElementById('miBoton');
```

```
function miFuncionClick() {  
    console.log('Se hizo clic!');  
}
```

```
boton.addEventListener('click', miFuncionClick); // Adjunto el oyente
```

```
// ... más tarde en el código ...
```

```
boton.removeEventListener('click', miFuncionClick); // Elimino el oyente
```

```
const boton = document.getElementById('miBoton');
```

```
function miFuncionClick() {  
  console.log('Se hizo clic!');  
}
```

```
boton.addEventListener('click', miFuncionClick); // Adjunto el oyente
```

```
// ... más tarde en el código ...
```

```
boton.removeEventListener('click', miFuncionClick); // Elimino el oyente
```

Permite controlar la fase de propagación del evento: `addEventListener` tiene un tercer parámetro opcional que te permite decidir si la función se ejecuta durante la "fase de captura" o la "fase de burbujeo". Esto es avanzado, pero muy útil para manejar la jerarquía de eventos en elementos anidados.

Sintaxis de `addEventListener`

La sintaxis básica es la siguiente:

```
elemento.addEventListener(evento, funcion, opciones);
```

- **elemento:** El objeto del DOM al que quieres adjuntar el evento (por ejemplo, un botón, un div, el window).
- **evento:** Un string que especifica el tipo de evento a escuchar (por ejemplo, 'click', 'mouseover', 'keydown', 'submit').
- **funcion:** La función que se ejecutará cuando ocurra el evento. A menudo se usa una función anónima o de flecha.
- **opciones (opcional):** Un objeto que permite configurar el oyente, por ejemplo, { once: true } para que la función se ejecute solo una vez.

Ejemplo práctico

Imagina que tienes un botón y quieres que al hacer clic en él, un párrafo cambie de color.

HTML:

```
<button id="cambiarColorBtn">Cambiar Color</button>
```

```
<p id="parrafo">Este es un párrafo que cambiará de color.</p>
```

JavaScript:

```
// 1. Obtener los elementos del DOM
```

```
const boton = document.getElementById('cambiarColorBtn');
```

```
const parrafo = document.getElementById('parrafo');
```

```
// 2. Adjuntar un oyente de evento 'click' al botón
```

```
boton.addEventListener('click', function() {
```

```
    // 3. Dentro de la función, se define qué hacer cuando se hace clic
```

```
    parrafo.style.color = 'blue';
```

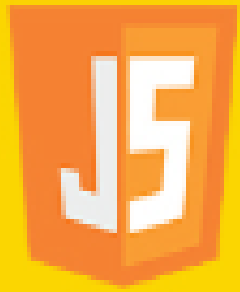
```
    parrafo.style.fontWeight = 'bold';
```

```
});
```

En este ejemplo, `addEventListener` es la herramienta clave que conecta la acción del usuario (el clic) con la respuesta del programa (el cambio de color). Sin él, el botón no tendría ninguna funcionalidad.

Espero que esta explicación detallada te aclare para qué sirve `addEventListener` y por qué es tan fundamental en el desarrollo web moderno.

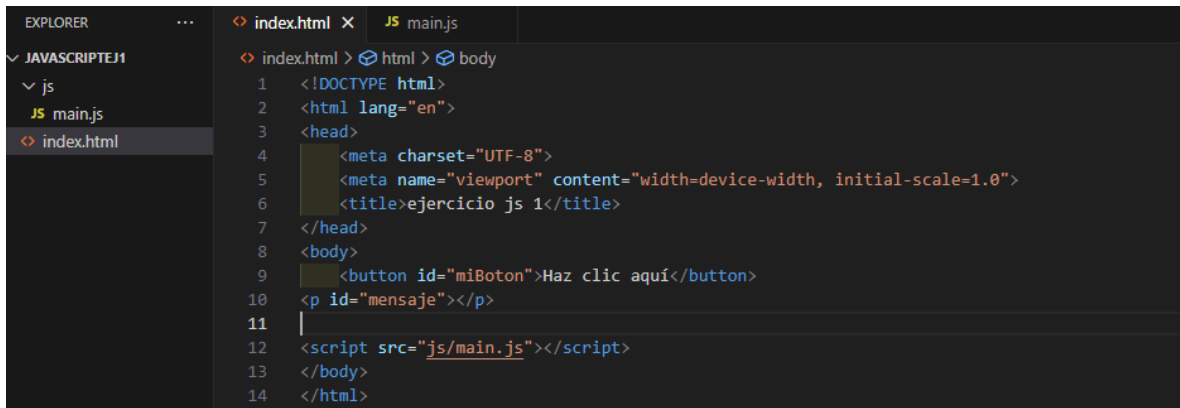
Ejercicios



{JavaScript}

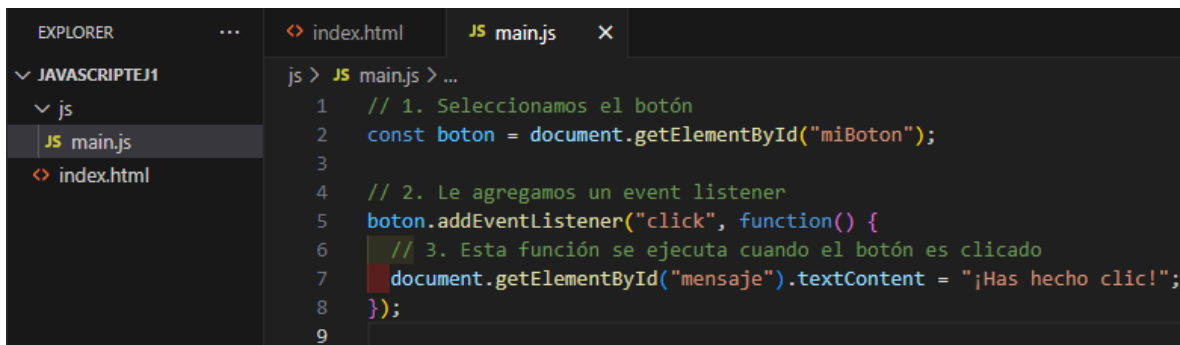
1. Ejercicio Escuchar un clic en un botón

Crear pagina html en la carpeta raíz javascriptej1



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>ejercicio js 1</title>
7 </head>
8 <body>
9   <button id="miBoton">Haz clic aquí</button>
10  <p id="mensaje"></p>
11
12  <script src="js/main.js"></script>
13 </body>
14 </html>
```

2 crear carpeta js en la carpeta raíz llamada js, dentro de ella crear el archivo main.js

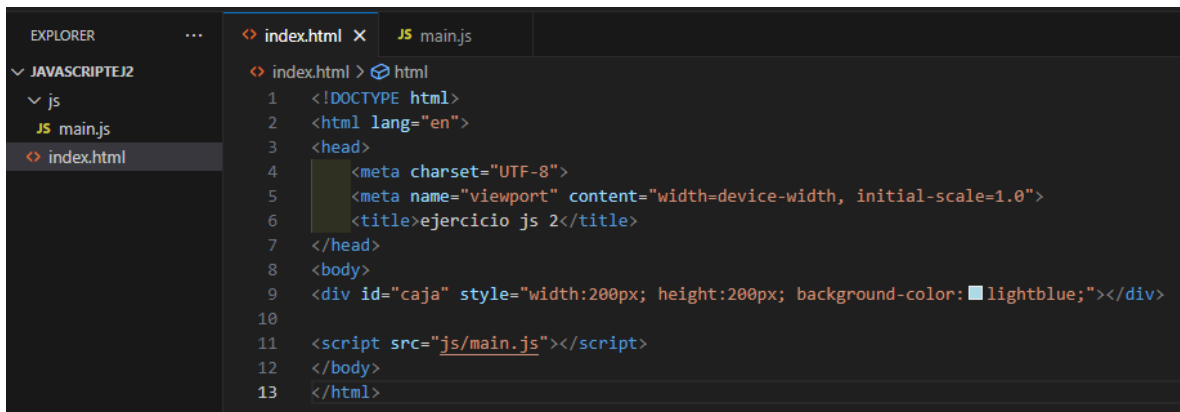


```
js > JS main.js > ...
1 // 1. Seleccionamos el botón
2 const boton = document.getElementById("miBoton");
3
4 // 2. Le agregamos un event listener
5 boton.addEventListener("click", function() {
6   // 3. Esta función se ejecuta cuando el botón es clicado
7   document.getElementById("mensaje").textContent = "¡Has hecho clic!";
8 });
9
```

- Se selecciona el botón con getElementById.
- Se usa addEventListener para escuchar el evento click.
- Se muestra un mensaje en un párrafo cuando se hace clic.

Ejercicio 2 Cambiar el color de fondo al pasar el ratón

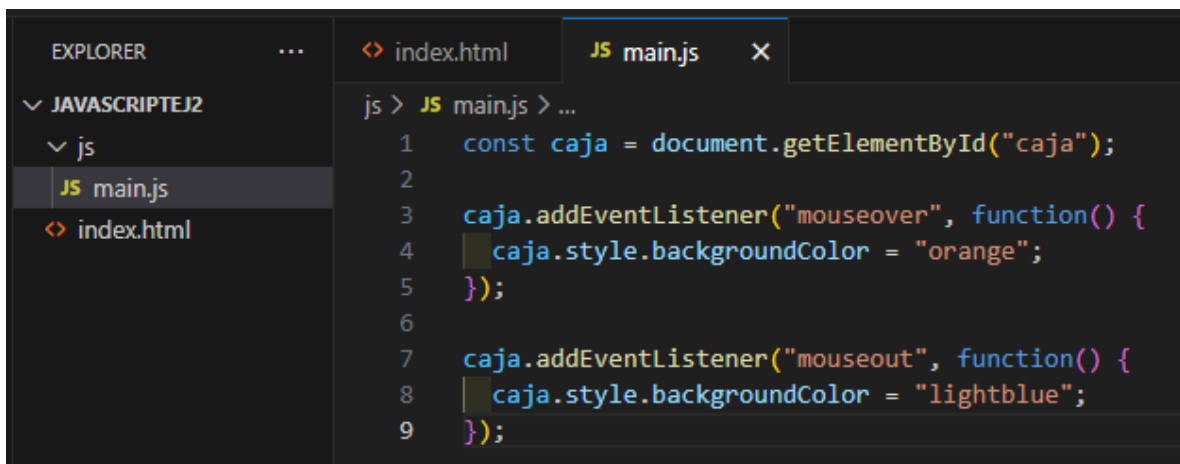
Carpeta raíz javascript2 archivo index.html



The screenshot shows the VS Code interface with the Explorer sidebar on the left. The project 'JAVASCRIPTEJ2' is expanded, showing a 'js' folder and the 'index.html' file. The 'index.html' file is open in the editor, displaying the following HTML code:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>ejercicio js 2</title>
7 </head>
8 <body>
9   <div id="caja" style="width:200px; height:200px; background-color:lightblue;"></div>
10
11   <script src="js/main.js"></script>
12 </body>
13 </html>
```

Carpeta llamada js dentro de la carpeta raíz



The screenshot shows the VS Code interface with the Explorer sidebar on the left. The project 'JAVASCRIPTEJ2' is expanded, showing a 'js' folder and the 'index.html' file. The 'main.js' file is open in the editor, displaying the following JavaScript code:

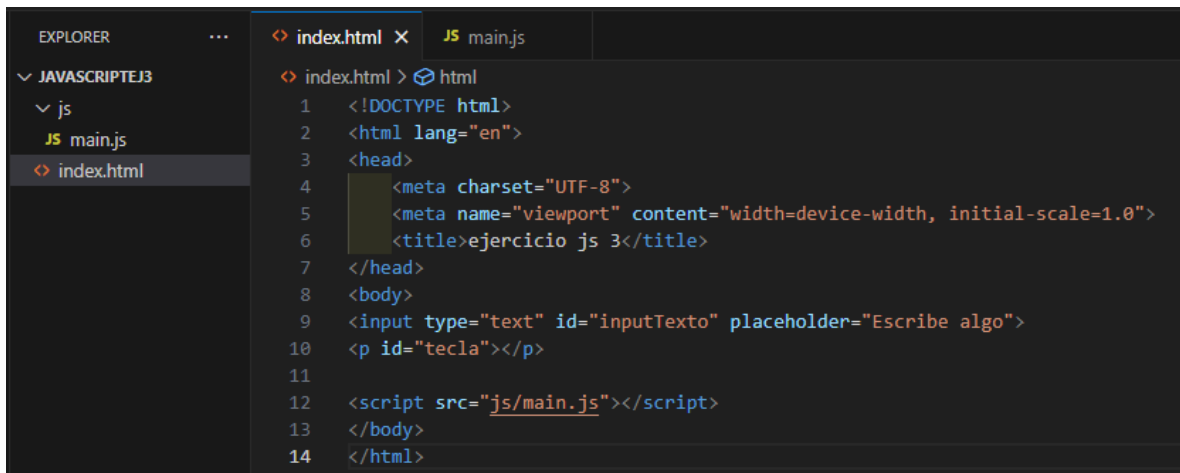
```
js > JS main.js > ...
1 const caja = document.getElementById("caja");
2
3 caja.addEventListener("mouseover", function() {
4   caja.style.backgroundColor = "orange";
5 });
6
7 caja.addEventListener("mouseout", function() {
8   caja.style.backgroundColor = "lightblue";
9 });
```

Este ejemplo usa dos eventos:

- mouseover: cuando el mouse entra al área del div.
- mouseout: cuando el mouse sale del área del div.

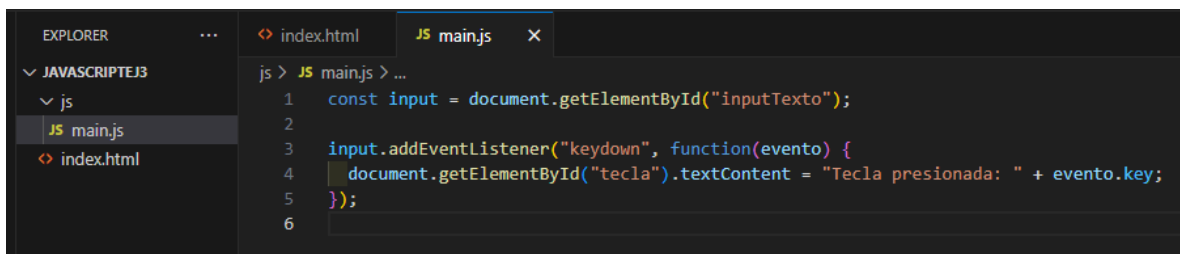
Ejercicio 3 Detectar teclas presionadas

Carpeta raíz javascript3 archivo index.html



```
index.html < html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>ejercicio js 3</title>
7 </head>
8 <body>
9   <input type="text" id="inputTexto" placeholder="Escribe algo">
10  <p id="tecla"></p>
11
12  <script src="js/main.js"></script>
13 </body>
14 </html>
```

Carpeta llamada js dentro de la carpeta raíz



```
js > JS main.js > ...
1 const input = document.getElementById("inputTexto");
2
3 input.addEventListener("keydown", function(evento) {
4   document.getElementById("tecla").textContent = "Tecla presionada: " + evento.key;
5 });
6
```

Aquí usamos el evento `keydown` para detectar cuando el usuario presiona una tecla dentro del input.